

# BPMN 2.0 Guide for Business Analysts and Process Architects

Business Process Model and Notation (BPMN) 2.0 is the industry standard for graphically modeling business processes. It defines **Flow Objects** (events, activities, gateways), **Connecting Objects** (sequence flows, message flows, associations), **Swimlanes** (pools, lanes) and **Artifacts** (data objects, groups, annotations) <sup>1</sup> <sup>2</sup>. Flow objects are the active elements: **Events** (circles) denote something that happens (e.g. *Start*, *End* or *Intermediate* events) <sup>3</sup>; **Activities** (rounded rectangles) denote work that is done (tasks, subprocesses) <sup>4</sup>; and **Gateways** (diamonds) control forking/merging of flows based on conditions (e.g. exclusive/XOR, parallel, inclusive) <sup>5</sup>. Sequence flows (solid arrows) show the order of execution <sup>6</sup>, message flows (dashed arrows) show communication across pools <sup>6</sup>, and associations (dotted lines) link artifacts or annotations to flow objects. **Pools** (swimlanes) represent major participants (e.g. departments or organizations) and can be drawn **expanded** (showing lanes and internal flow) or **collapsed** (just a container outline) <sup>7</sup>. **Lanes** partition a pool by function or role; they contain flow objects and indicate responsibility <sup>8</sup>.

## Rules Governing Interactions

BPMN imposes rules to ensure **well-formed** models. Sequence flows may only connect **flow nodes** (events, activities, gateways) within the same process/pool <sup>9</sup>; message flows connect elements in *different* pools <sup>10</sup>. **Pools cannot be connected by sequence flows**, only by message flows to show cross-organizational exchanges. A **Start Event** must have no incoming flows (it **creates** a token) and an **End Event** must have no outgoing flows (it **terminates** a token) <sup>11</sup>. By convention every process should explicitly include at least one start and one end event for clarity <sup>11</sup>. Gateways that split flows should have corresponding join gateways (or implicit merges) to preserve logical consistency; for example, an XOR-split is often paired with an XOR-join, and all flows must reconverge before process completion (avoiding “loose ends”). Pools each contain a single process definition (or a “private process”); they may not share sequence flows with other pools. Lanes are purely organizational (no semantic effect on flows). In practice, BPMN tooling often enforces well-formedness rules (even though the OMG spec defines many rules only informally) <sup>12</sup>.

## Common Modeling Patterns

- **Exclusive (XOR) vs. Parallel (AND) Gateways:** Use an *exclusive gateway* (X in diamond) to model a decision where only one outgoing branch is taken, based on conditions <sup>5</sup>. For example, in an approval process an exclusive gateway might branch on “Approved” vs. “Rejected”. A *parallel gateway* (+ in diamond) splits into concurrent branches: all outgoing flows are taken. For instance, after an order is approved you might **parallelize** “Notify Customer” and “Arrange Shipment” tasks to run at the same time. (An *inclusive gateway* can be used when multiple, but not necessarily all, branches may be taken.) When joining, use matching gateways: parallel joins wait for tokens on all incoming flows; exclusive joins merge multiple paths (or omit an explicit join if only one path arrives).
- **Sub-Processes:** Use *sub-processes* (rounded box with “+” marker) to encapsulate detailed process logic. A collapsed sub-process hides its internal flow and is invoked like a single task; an expanded

sub-process shows its own start/end events and flow objects inside <sup>13</sup>. Sub-processes should have their own start and end events and cannot have parent-sequence-flows crossing their boundary <sup>13</sup>. *Call Activities* reference a global (reusable) process definition. Sub-processes help manage complexity by breaking a large process into nested pieces.

- **Boundary Events (Error and Escalation):** Attach *boundary events* to tasks or sub-processes to handle exceptions. For example, an **Error Boundary Event** (a circle with lightning icon) interrupts the activity on error and directs flow to an error-handler path <sup>14</sup>. An **Escalation Boundary Event** signals a non-critical condition up to a higher-level scope <sup>15</sup>. Escalations are *non-interrupting* by default: the normal flow continues and an escalation token is thrown toward the parent process <sup>16</sup> <sup>17</sup>. (In contrast, error boundary events are typically *interrupting*: the original activity halts and the error path is taken <sup>14</sup>.) Inside sub-processes, **Escalation Events** can be thrown (or caught) similarly to signal business deviations without aborting the entire process <sup>16</sup> <sup>18</sup>.
- **Error Flows and Compensation:** A thrown *error event* (via an intermediate or end error event) is caught by the nearest matching error handler (boundary event or event subprocess) <sup>18</sup>. The catching error path is chosen by matching an `errorCode`. If uncaught, the error propagates up to callers (or aborts the process) <sup>18</sup>. In contrast, a *compensation event* (circle with rewind arrow) is used to trigger compensating activities for already-completed work (not covered here, but often attached to end or boundary positions). BPMN provides rich exception handling: use error events for critical failures, escalations for non-critical alerts, and optionally compensation for rollbacks.

## Conformance Levels and Constraints

The BPMN 2.0 standard (ISO/IEC 19510:2013) defines multiple **conformance classes** for tools and models <sup>19</sup>. Key classes include:

| Conformance Class             | Description                                                                                                                       |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Process Modeling</b>       | Supports all flow-graph elements in descriptive, analytical, and executable notations (diagram elements and data) <sup>19</sup> . |
| <b>Process Execution</b>      | Supports importing and executing process diagrams according to the BPMN semantic meta-model <sup>20</sup> .                       |
| <b>BPEL Process Execution</b> | Supports mapping BPMN models to BPEL for Web-service orchestration <sup>21</sup> .                                                |
| <b>Choreography Modeling</b>  | Supports BPMN choreography diagrams (multi-party interactions) with full element semantics <sup>21</sup> .                        |

In addition to these, **well-formedness constraints** govern model correctness. The OMG spec describes rules informally <sup>12</sup>, so practitioners rely on guidelines and tool validation. For example, a well-formed process typically has exactly one start event per triggered flow (or multiple if modeling parallel triggers), and at least one end event. Flows should **not** be dangling: every activity (except start/end) should have incoming and outgoing sequence flows. Gateways should *balance* logic (avoid ambiguous merges). In practice, many BPMN modelers recommend explicitly drawing all start/end events and using matching splits/joins <sup>11</sup> <sup>12</sup>. Completeness also means the diagram captures all described steps; any missing step in the narrative indicates the BPMN is incomplete. (Since the standard omits formal OCL rules, the community-driven approach is to follow these modeling conventions for correctness <sup>12</sup>.)

## BPMN XML Serialization

A BPMN model is persisted as XML conforming to the BPMN 2.0 XSD. The root is `<definitions>`, typically with the BPMN namespace `xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL"` and a `targetNamespace` <sup>22</sup>. Inside, a `<process>` element (with `id` and optional `name`) contains the flow. Flow elements are represented by XML tags matching their graphical symbols: `<startEvent>`, `<task>` or subtype (`<userTask>`, `<serviceTask>`, etc.), `<gateway>` (e.g. `<exclusiveGateway>`, `<parallelGateway>`), and `<endEvent>`. Each flow node has a unique `id` attribute. Connections are encoded as `<sequenceFlow id="..." sourceRef="..." targetRef="..." />`, linking the source and target element IDs. For example:

```
<definitions id="definitions"
  xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL"
  targetNamespace="http://flowable.org/bpmn20">
  <process id="financialReport" name="Monthly financial report process">
    <startEvent id="theStart"/>
    <sequenceFlow id="flow1" sourceRef="theStart" targetRef="writeReportTask"/>
    <userTask id="writeReportTask" name="Write monthly financial report">
      <potentialOwner>
        <resourceAssignmentExpression>
          <formalExpression>accountancy</formalExpression>
        </resourceAssignmentExpression>
      </potentialOwner>
    </userTask>
    <sequenceFlow id="flow2" sourceRef="writeReportTask"
targetRef="verifyReportTask"/>
    <userTask id="verifyReportTask" name="Verify monthly financial report">
      <potentialOwner>
        <resourceAssignmentExpression>
          <formalExpression>management</formalExpression>
        </resourceAssignmentExpression>
      </potentialOwner>
    </userTask>
    <sequenceFlow id="flow3" sourceRef="verifyReportTask" targetRef="theEnd"/>
    <endEvent id="theEnd"/>
  </process>
</definitions>
```

Each `<userTask>` (or other activity) can include nested extensions (assignments, scripts, etc.), and the `<definitions>` may contain BPMN Diagram Interchange (`<BPMNDiagram>`) elements for layout (not shown here). When modeling collaborations, a `<collaboration>` element with `<participant>` elements (referring to process IDs) and `<messageFlow>` definitions is used. The above snippet (adapted from Flowable documentation <sup>22</sup> <sup>23</sup>) illustrates the core structure: definitions → process → events/tasks/gateways → sequenceFlow links.

# AI Mapping from Text to BPMN

Modern AI and NLP techniques can assist in converting natural-language process descriptions into formal BPMN models. Research shows that extracting *process elements* from text involves techniques like **dependency parsing** (to identify sentence structure and core actions), **named-entity recognition** (to find business entities and roles), and **subject-verb-object (SVO)** analysis <sup>24</sup>. Neural methods (transformer LLMs) can then learn mappings from text to BPMN notation. For example, specialized models like *TXT2BPMN* (fine-tuned on 30,000 text-diagram pairs) automate generating BPMN flows from descriptions <sup>25</sup>. Commercial tools (e.g. Bonita's AI BPMN generator) advertise: “Turn a process description into a full BPMN model” using NLP <sup>26</sup>.

**Challenges:** Natural language is inherently ambiguous and context-dependent. AI systems must address *alias detection* (different words referring to the same actor or concept) and *anaphora resolution* (pronouns or references linking back to earlier entities) <sup>27</sup>. They must correctly identify and categorize activities, events, and decision points, then place them into appropriate BPMN constructs while preserving the process logic <sup>28</sup>. Crucially, the AI must maintain the **semantic integrity** of the original description: the output BPMN should faithfully reflect the intended flow of the narrative without adding or losing meaning <sup>29</sup>.

**Limitations:** Current models often struggle with complex or technical domain language, long multi-step descriptions, and nested subprocess structures <sup>30</sup> <sup>29</sup>. They may default to common patterns and miss rare exceptions. Dataset bias (trained examples not covering all industries) can lead to incomplete or inaccurate models <sup>30</sup>. Moreover, conditional logic and parallelism implied in text can be hard to infer without explicit cues. Therefore, human oversight remains essential: generated BPMN should be reviewed for correctness and clarity. Despite these challenges, AI-assisted tools can greatly speed up initial modeling of neutral, cross-industry processes (e.g. **onboarding, approval workflows, incident response, order-to-cash**) and act as a valuable starting point for analysts. By combining domain expertise with AI-generated drafts, teams can achieve implementation-ready BPMN models more efficiently.

**Sources:** Standard BPMN 2.0 definitions and best practices (OMG BPMN 2.0, ISO 19510) <sup>31</sup> <sup>12</sup>; BPMN element descriptions (Wikipedia, WebRatio, Flowable docs) <sup>32</sup> <sup>5</sup> <sup>22</sup>; modeling guidelines (Camunda Docs) <sup>11</sup> <sup>14</sup>; AI/NLP research on text-to-BPMN <sup>27</sup> <sup>30</sup>; examples from Flowable and Bonita documentation <sup>22</sup> <sup>26</sup>.

---

<sup>1</sup> <sup>13</sup> <sup>32</sup> Getting started with BPM | WebRatio

<https://my.webratio.com/learn/learningobject/getting-started-with-bpm>

<sup>2</sup> <sup>3</sup> <sup>4</sup> <sup>5</sup> <sup>6</sup> <sup>7</sup> <sup>8</sup> <sup>9</sup> <sup>10</sup> Business Process Model and Notation - Wikipedia

[https://en.wikipedia.org/wiki/Business\\_Process\\_Model\\_and\\_Notation](https://en.wikipedia.org/wiki/Business_Process_Model_and_Notation)

<sup>11</sup> Creating readable process models | Camunda 8 Docs

<https://docs.camunda.io/docs/components/best-practices/modeling/creating-readable-process-models/>

<sup>12</sup> [1502.06297] Elements of style of BPMN language

<https://arxiv.org/abs/1502.06297>

<sup>14</sup> <sup>18</sup> Error events | Camunda 8 Docs

<https://docs.camunda.io/docs/components/modeler/bpmn/error-events/>

15 16 17 Escalation events | Camunda 8 Docs

<https://docs.camunda.io/docs/components/modeler/bpmn/escalation-events/>

19 20 21 Business Process Modeling and Notation (BPMN) 101 | Smartsheet

[https://www.smartsheet.com/beginners-guide-business-process-modeling-and-notation-bpmn?srltid=AfmBOophEErYbxjZwxr8cTVWMV6fHZk\\_6pBF3JzywqyUu4ODf8IySBC](https://www.smartsheet.com/beginners-guide-business-process-modeling-and-notation-bpmn?srltid=AfmBOophEErYbxjZwxr8cTVWMV6fHZk_6pBF3JzywqyUu4ODf8IySBC)

22 23 BPMN 2.0 Introduction · Flowable Open Source Documentation

<https://www.flowable.com/open-source/docs/bpmn/ch07a-BPMN-Introduction>

24 27 28 29 30 From Dialogue to Diagram: Task and Relationship Extraction from Natural Language for Accelerated Business Process Prototyping

<https://arxiv.org/html/2312.10432v1>

25 TXT2BPMN · Models · Dataloop

[https://dataloop.ai/library/model/fachati\\_txt2bpmn/](https://dataloop.ai/library/model/fachati_txt2bpmn/)

26 Bonita AI BPMN Generator | Create BPMN Diagrams from Text

<https://www.bonitasoft.com/ai-bpmn-generator>

31 Business Process Model and Notation (BPMN)

<https://www.trisotech.com/bpmn/>